UNIT – 1

ASSESSMENT – 2

PYTHON CODE

NIHARIKA PREEJESH

192524506

1. Emergency Medicine Delivery Using A* Search Algorithm

```
import heapq

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('D', 3), ('E', 5)],
    'C': [('F', 2)],
    'D': [('G', 4)],
    'E': [('G', 2)],
    'F': [('G', 3)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 5,
    'D': 4,
    'E': 2,
```

```
    'F': 3,
    'G': 0
}

def astar(start, goal):
    pq = []
    heapq.heappush(pq, (0, start))

    cost = {start: 0}
    parent = {start: None}

    while pq:
        f, node = heapq.heappop(pq)

        if node == goal:
            break

        for neighbor, weight in graph[node]:
            new_cost = cost[node] + weight

            if neighbor not in cost or new_cost < cost[neighbor]:
                cost[neighbor] = new_cost
                priority = new_cost + heuristic[neighbor]
                heapq.heappush(pq, (priority, neighbor))
```

```
                parent[neighbor] = node

    path = []
    current = goal

    while current:
        path.append(current)
        current = parent[current]

    path.reverse()

    print("Optimal Path:", " -> ".join(path))
    print("Total Cost:", cost[goal])

astar('A', 'G')
```

2. Traffic Signal Optimization Using Genetic Algorithm

```
import random

# Initial traffic signal timings (seconds)
signals = [30, 45, 60, 50]
```

```
def fitness(solution):
    # Lower total waiting time is better
    return sum(solution)

best = signals.copy()

for _ in range(10):
    new = best.copy()

    index = random.randint(0, len(new)-1)
    new[index] += random.choice([-5, 5])

    if fitness(new) < fitness(best):
        best = new

print("Optimized Signal Timings:", best)
print("Fitness Value:", fitness(best))
```

3. Mars Rover Navigation Using an Online Search Agent

```
terrain = ["Safe", "Rock", "Safe", "Crater", "Safe"]

for i, area in enumerate(terrain):
    print("Location", i + 1)
```

```
    if area == "Safe":
        print("Move Forward")
    else:
        print("Hazard Detected:", area)
        print("Choose Alternate Path")

    print()
```

4. Exam Timetable Generation Using Constraint Satisfaction Problem (CSP)

```
subjects = ["Math", "Physics", "Chemistry"]
slots = ["Morning", "Afternoon"]

assignment = {}

def backtrack(index):
    if index == len(subjects):
        return True

    subject = subjects[index]

    for slot in slots:
```

```
        if slot not in assignment.values():
            assignment[subject] = slot

            if backtrack(index + 1):
                return True

            del assignment[subject]

    return False

if backtrack(0):
    print("Exam Timetable:")
    for subject, slot in assignment.items():
        print(subject, "->", slot)
else:
    print("No valid timetable found.")
```

5. Game AI Using Minimax and Alpha-Beta Pruning

```
def minimax(depth, node, maximizing, values, alpha, beta):

    if depth == 3:
        return values[node]
```

```
    if maximizing:
        best = float('-inf')

        for i in range(2):
            value = minimax(depth + 1, node * 2 + i, False, values, alpha,
beta)
            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = float('inf')

        for i in range(2):
            value = minimax(depth + 1, node * 2 + i, True, values, alpha,
beta)
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
```

```
            break

    return best


values = [3, 5, 6, 9, 1, 2, 0, -1]

result = minimax(0, 0, True, values, float('-inf'), float('inf'))

print("Best Value:", result)
```